

DESIGN AND ANALYSIS OF ALGORITHMS

(23CS501 · R23 Regulation · B.Tech III Year I Semester)

Department of Computer Science and Engineering

UNIT – III : THE GREEDY METHOD

Syllabus Coverage

- The General Method
- Knapsack Problem (Fractional Knapsack)
- Job Sequencing with Deadlines
- Minimum-cost Spanning Trees — Prim's Algorithm, Kruskal's Algorithm
- An Optimal Randomized Algorithm
- Optimal Merge Patterns
- Single Source Shortest Paths (Dijkstra's Algorithm)

How to use these notes

Each topic follows the same rhythm: an intuitive real-life analogy, the formal idea, worked numeric examples, a diagram, pseudocode, complexity analysis, common pitfalls, and exam-style practice questions. Read the analogy first — it is designed to make the algorithm feel obvious before you see the formal version. These notes are written to be read aloud or dictated for revision, with every topic covered in full depth.

1. The Greedy Method — General Approach

1.1 What does “Greedy” mean in algorithm design?

A greedy algorithm builds up a solution piece by piece, and at every single step it makes the choice that looks best right now — the locally optimal choice — without ever going back to reconsider an earlier decision. It never worries about the future consequences of today's choice; it simply grabs whatever is most attractive at the moment and moves on. This is fundamentally different from approaches that explore multiple possibilities and backtrack (like Backtracking) or that solve and remember many overlapping sub-problems (like Dynamic Programming).

Real-life analogy

Imagine you are given a large pile of coins and told to pick handfuls to make change of ₹78, using the fewest coins possible, and you are allowed to look at only one coin at a time. A greedy shopper always picks the largest coin that does not overshoot the target: first a ₹50 coin, then a ₹20, then a ₹5, then a ₹2, then a ₹1 — done. At no point did the shopper think about what coins might be picked next; each pick was simply the best available one at that instant. This is exactly the greedy philosophy — myopic, one step at a time, never looking back.

The catch is that the local best choice does not always lead to the global best solution. Greedy algorithms are fast and simple, but they only produce a truly optimal answer for problems that have two specific mathematical properties, described in Section 1.3. When a problem lacks these properties, greedy gives a quick approximate answer, not necessarily the best one — for example, with certain irregular coin denominations (like 1, 3, 4), greedily making change for 6 gives 4+1+1 (three coins) when 3+3 (two coins) is actually better. This single counter-example is why every greedy algorithm needs a proof before it can be trusted.

1.2 The Control Abstraction

Every greedy algorithm can be described using the same skeleton, regardless of the specific problem it solves. This generic template is called the control abstraction for the greedy method:

```
Algorithm Greedy(a[1..n])
// a[1..n] contains the n candidate inputs (activities, items, edges...)
{
    solution := empty set;           // the set we build up
    for i := 1 to n do
    {
        x := select(a);             // pick the best remaining
        candidate
        if feasible(solution, x) then
            solution := solution + {x}; // accept x only if it keeps the
                                       // partial solution valid
        }
    }
    return solution;
}
```

The three functions `select()`, `feasible()`, and an implicit `objective()` are what change from problem to problem. `select()` defines the greedy criterion — e.g. 'largest profit-to-weight ratio' in Knapsack, 'smallest edge weight' in Kruskal's, 'earliest deadline' influence in Job Sequencing; `feasible()` is a Boolean test that checks whether adding the new element still respects the problem's constraints (e.g. does it exceed the knapsack capacity? does it create a

cycle?); and `objective()` measures how good the final solution is (total profit, total cost, etc.), which is what we are ultimately trying to optimise. Designing a greedy algorithm for a new problem is, in essence, the design of a correct and provably-optimal `select()` rule.

1.3 Two Properties a Problem Must Have for Greedy to be Optimal

(a) Greedy-choice property

A globally optimal solution can always be reached by making a sequence of locally optimal (greedy) choices. In other words, it is always safe to grab the best-looking option right now — you will never regret it later, and no alternative first choice could ever lead to a strictly better final answer. This must be proved for each new problem using an 'exchange argument': assume some optimal solution does NOT include the greedy choice, then show you can always modify it to include the greedy choice without making it any worse. It is not automatically true for every problem.

(b) Optimal substructure

A problem exhibits optimal substructure if an optimal solution to the whole problem contains, within it, optimal solutions to smaller sub-problems. After the greedy choice is made, what remains is a smaller version of the same problem, and solving that smaller version optimally (and combining it with the choice already made) gives an optimal solution to the original problem. This property is also shared by Dynamic Programming problems — the difference is that DP problems typically do not also have the greedy-choice property, so DP must try many sub-problems, while greedy problems can commit to just one.

Why this matters

If a problem has both properties, greedy is not just fast — it is provably correct, and usually the simplest possible algorithm for the problem (e.g. Prim's, Kruskal's, Dijkstra's, Huffman coding, Fractional Knapsack, Job Sequencing). If a problem lacks the greedy-choice property (like 0/1 Knapsack, where you cannot take a fraction of an item, or the Travelling Salesperson Problem), a greedy pick can permanently lock you out of the true optimum, and Dynamic Programming, Backtracking, or Branch-and-Bound must be used instead.

1.4 Characteristics of Problems Solvable by the Greedy Method

Not every optimisation problem yields to a greedy strategy. Problems that do tend to share these characteristics:

- The problem can be viewed as making a sequence of decisions, one candidate at a time, drawn from a finite set of candidates.
- There is a simple, easily-computable selection criterion that ranks candidates (e.g. a ratio, a weight, a deadline) which correlates directly with the objective being optimised.
- A feasibility check exists that can quickly verify whether adding a candidate keeps the partial solution valid, without needing to know anything about future choices.
- The problem's optimal solution can be built incrementally, and once a choice is locked in, it never needs to be undone.

1.5 Greedy vs Dynamic Programming — Quick Comparison

Aspect	Greedy Method	Dynamic Programming
Choice made	One irrevocable choice per step	Explores overlapping sub-problems, may reconsider
Sub-problems solved	Only one path is ever followed	Many; results are memoized/tabulated
Speed	Usually faster, simpler code	Usually slower, needs extra memory (table)
Optimality	Optimal only if greedy-choice property holds	Always optimal if the recurrence is correct
Typical examples	Prim's, Kruskal's, Dijkstra's, Job Sequencing	0/1 Knapsack, Matrix Chain, LCS
Proof technique	Exchange argument / matroid theory	Principle of optimality (Bellman)

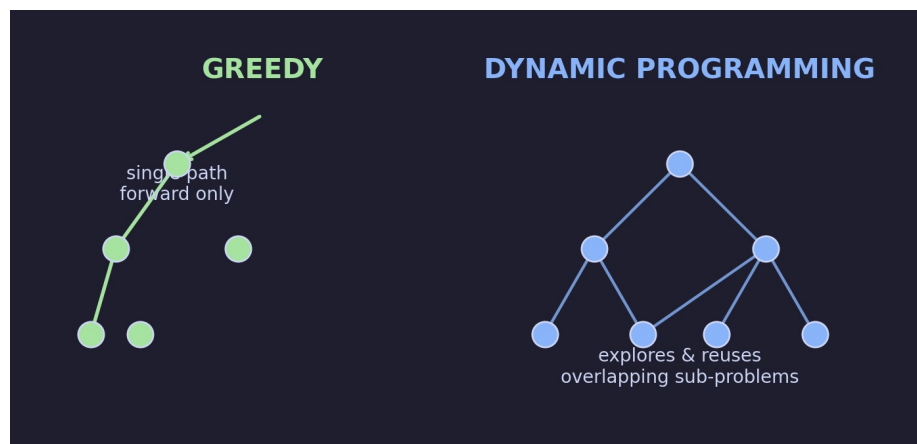


Figure 1.1 — Greedy follows a single forward path; DP explores and reuses overlapping sub-problems.

1.6 Worked Illustration: Activity/Interval Selection

A classic first example of the greedy method (useful for building intuition before the syllabus topics) is the Activity Selection Problem: given n activities each with a start time and a finish time, and a single resource (e.g. one lecture hall) that can host only one activity at a time, select the maximum possible number of non-overlapping activities.

Consider 6 activities with (start, finish) times: A1(1,4), A2(3,5), A3(0,6), A4(5,7), A5(3,9), A6(6,10).

Step	Activity considered (sorted by finish time)	Decision	Reason
1	A1 (1,4)	Select	First activity by finish time; always safe to take
2	A2 (3,5)	Reject	Starts at 3, before A1 finishes at 4 — overlaps
3	A4 (5,7)	Select	Starts at 5, at/after A1's finish (4) — no overlap
4	A6 (6,10)	Reject	Starts at 6, before A4 finishes at 7 — overlaps

Result and Insight

Selected activities: {A1, A4} — 2 non-overlapping activities is the maximum possible here. The greedy rule is: always sort by finish time (not start time, and not duration) and always pick the next activity whose start time is not earlier than the finish time of the last selected activity. Sorting by finish time is the crucial insight — it greedily reserves the smallest possible 'block' of the resource at each step, leaving maximum room for future activities.

1.7 Common Pitfalls and Exam Tips

- Do not assume a greedy strategy works just because it 'feels natural' — always check for a known counter-example or require a proof sketch in your answer.
- Remember that sorting is almost always the first step of a greedy algorithm — this is why most greedy algorithms have an $O(n \log n)$ lower bound on their running time.
- In exams, always explicitly state the greedy-choice property and optimal substructure when asked to justify why a greedy algorithm is correct — this is frequently a separate mark.
- Distinguish 'greedy gives a good approximate answer' (e.g. for 0/1 Knapsack, NP-hard problems) from 'greedy gives the exact optimal answer' (e.g. for Fractional Knapsack, MST, Dijkstra's, Huffman coding) — many exam questions test this distinction directly.

1.8 Practice Questions

- Define the Greedy method and explain the control abstraction with the roles of select(), feasible(), and objective().
- State and explain the greedy-choice property and optimal substructure property with one example each.
- Give an example of a problem where the greedy method fails to give an optimal solution, and explain why.

- Compare and contrast the Greedy method with Dynamic Programming, listing at least four points of difference.

2. The Fractional Knapsack Problem

2.1 Problem Statement

You are given n items, each with a weight $w[i]$ and a profit (value) $p[i]$, and a knapsack (bag) that can carry a maximum total weight M . Unlike the 0/1 version of this problem, here you are allowed to take a fraction $x[i]$ of any item, where $0 \leq x[i] \leq 1$. The goal is to choose the fractions $x[i]$ so that the total weight does not exceed M and the total profit is as large as possible. Formally:

Maximise $\sum (x[i] \cdot p[i])$ subject to $\sum (x[i] \cdot w[i]) \leq M$ and $0 \leq x[i] \leq 1$ for all i .

Real-life analogy

Picture a thief with a sack that can hold exactly 50 kg, standing in a shop that sells gold dust, silver dust, and bronze dust — all sold loose by weight, so any fraction can be scooped up. Obviously the thief scoops purely gold dust first (highest value per kg), then silver, then bronze, until the sack is exactly full. Because the goods are loose powder, there is no penalty for taking a partial amount — this is precisely why the greedy strategy works perfectly here, but would fail if the items were indivisible objects like sealed jewellery boxes.

2.2 The Greedy Strategy: Sort by Profit-to-Weight Ratio

The greedy rule is to compute the ratio $p[i] / w[i]$ for every item — this tells you how much profit you earn per unit of weight, i.e. how 'dense' the value of the item is. Sort all items in decreasing order of this ratio. Then fill the knapsack by taking items in that order: take the whole item if it fits, and if an item does not fit completely, take just enough of a fraction of it to exactly fill the remaining capacity, and stop.

This greedy choice is provably optimal for the fractional version. Intuitively, any solution that leaves 'room' unused while a higher-ratio item is still partially available cannot be optimal, because swapping in more of the higher-ratio item strictly increases profit without increasing weight beyond M — this is the exchange argument that proves the greedy-choice property for this problem.

2.3 Proof Sketch of Optimality

Suppose, for contradiction, that some optimal solution does not take items in decreasing ratio order — say it takes a fractional or zero amount of a higher-ratio item i while taking a full unit of a lower-ratio item j ($\text{ratio}[i] > \text{ratio}[j]$). We can always trade a small amount ϵ of item j for an equal weight ϵ of item i (this stays feasible since total weight is unchanged), and profit changes by $\epsilon \cdot (p[i]/w[i] - p[j]/w[j]) = \epsilon \cdot (\text{ratio}[i] - \text{ratio}[j]) > 0$. This means the supposedly optimal solution was not actually optimal — a contradiction. Hence, an optimal solution must always prefer higher-ratio items first, which is exactly what the greedy algorithm does.

2.4 Worked Example 1

Knapsack capacity $M = 20$. Three items are available:

Item	Weight $w[i]$	Profit $p[i]$	Ratio $p[i]/w[i]$
1	18	25	1.39
2	15	24	1.60
3	10	15	1.50

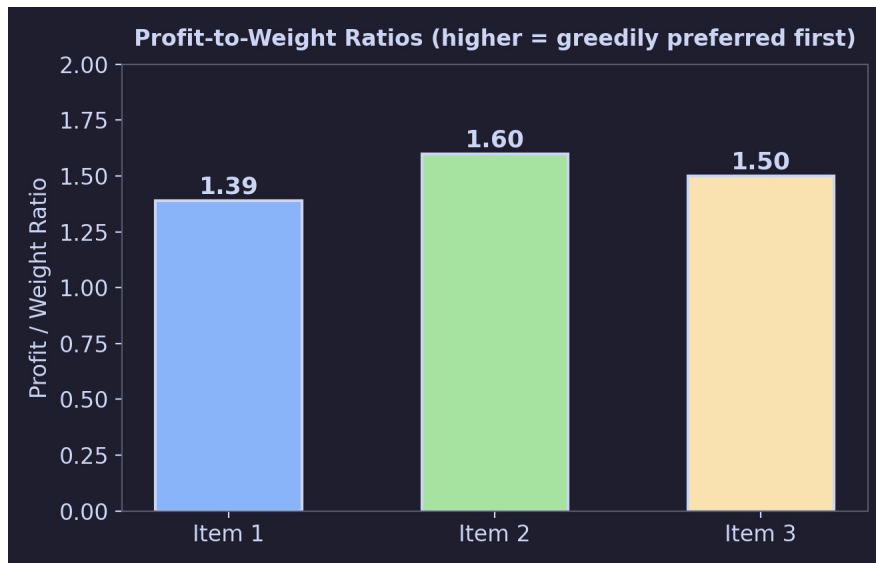


Figure 2.1 — Profit-to-weight ratios for the three items; Item 2 has the highest ratio and is taken first.

Step 1 — Sort items by ratio in decreasing order: Item 2 (1.60), Item 3 (1.50), Item 1 (1.39).

Step 2 — Fill greedily:

- Take Item 2 completely: weight used = 15, profit earned = 24. Remaining capacity = $20 - 15 = 5$.
- Item 3 needs 10 kg but only 5 kg of room remains, so take the fraction $x = 5/10 = 0.5$ of Item 3: profit earned = $0.5 \times 15 = 7.5$. Remaining capacity = 0.
- Item 1 is skipped entirely — no capacity left ($x = 0$).

Item taken	Fraction $x[i]$	Weight contributed	Profit contributed
Item 2	1.0	15	24
Item 3	0.5	5	7.5
Item 1	0.0	0	0

Result

Total weight used = $15 + 5 = 20$ (exactly fills the knapsack).

Total profit = $24 + 7.5 = 31.5$, and this is the maximum profit achievable — no other combination of

fractions can beat it.

2.5 Worked Example 2 (a second, independent case for practice)

Knapsack capacity $M = 60$. Five items:

Item	Weight $w[i]$	Profit $p[i]$	Ratio $p[i]/w[i]$
1	5	30	6.00
2	10	40	4.00
3	15	45	3.00
4	22	77	3.50
5	25	90	3.60

Sorted by ratio (decreasing): Item 1 (6.00), Item 2 (4.00), Item 5 (3.60), Item 4 (3.50), Item 3 (3.00).

Order	Item	Weight needed	Capacity before	Action	Profit earned
1	Item 1	5	60	Take fully → capacity becomes 55	30
2	Item 2	10	55	Take fully → capacity becomes 45	40
3	Item 5	25	45	Take fully → capacity becomes 20	90
4	Item 4	22	20	Take fraction $20/22 = 0.909$	70.0
5	Item 3	15	0	Take nothing ($x = 0$)	0

Result

Total profit = $30 + 40 + 90 + 70.0 + 0 = 230.0$ (the fraction-4 contribution is $0.909 \times 77 \approx 70.0$). This is the maximum achievable profit for $M = 60$ with these five items.

2.6 Pseudocode

```

Algorithm GreedyKnapsack(p[1..n], w[1..n], M)
// p[i], w[i] are profit and weight of item i, sorted so that
// p[1]/w[1] >= p[2]/w[2] >= ... >= p[n]/w[n]
{
    for i := 1 to n do x[i] := 0.0;
    U := M;                                // remaining capacity
    for i := 1 to n do
    {
        if (w[i] > U) then break;          // cannot even fit fully
        x[i] := 1.0;    U := U - w[i];
    }
    if (i <= n) then x[i] := U / w[i];    // take a fraction of the next
item
    return x[];
}

```

2.7 Complexity Analysis

Step	Time
Computing all ratios $p[i]/w[i]$	$O(n)$
Sorting items by ratio $p[i]/w[i]$	$O(n \log n)$
Greedy fill loop (single pass)	$O(n)$
Overall	$O(n \log n)$ — dominated by the sort

Important distinction to remember for exams

Fractional Knapsack (this section) is solved optimally by the Greedy Method because items can be split. 0/1 Knapsack — where each item must be taken whole or not at all — does NOT have the greedy-choice property (a locally best ratio can force a bad overall combination), so it needs Dynamic Programming instead. This contrast is a favourite short-answer exam question.

2.8 Applications

- Resource allocation where partial allocation is meaningful — bandwidth sharing, budget splitting across divisible investment options.
- Load balancing across servers when a task's work can be split fractionally across machines.
- Blending/mixing problems in operations research (e.g. blending grades of ore, oil, or grain to maximise value within a weight or volume limit).

2.9 Practice Questions

- Explain why the Fractional Knapsack problem satisfies the greedy-choice property while the 0/1 Knapsack problem does not.

- Given items with (weight, profit) pairs (2,10), (3,5), (5,15), (7,7), (1,6) and knapsack capacity 12, find the maximum profit using the greedy method (show all working).
- Derive the time complexity of the Fractional Knapsack greedy algorithm and explain which step dominates.

3. Job Sequencing with Deadlines

3.1 Problem Statement

There are n jobs, and each job i has a deadline $d[i]$ and a profit $p[i]$. Each job takes exactly one unit of time to complete, and only one job can be processed at any given time slot. A job earns its profit $p[i]$ only if it is completed on or before its deadline $d[i]$. The goal is to choose a subset of jobs — and an order/schedule for them — so that every scheduled job meets its deadline and the total profit collected is maximised.

Real-life analogy

Think of a freelance graphic designer who receives several project offers on the same day, each with its own one-day deadline (e.g. 'due by day 3') and its own payment. The designer can only work on one project per day slot and has no time machine, so a project accepted past its deadline is worthless. A smart designer first ranks jobs by how much they pay, then for each job (starting with the highest payer) tries to slot it into the latest available day at or before its deadline — this keeps the earlier days open for other jobs that might have even tighter deadlines. This is exactly the greedy job-sequencing strategy.

3.2 The Greedy Strategy

1. Sort all jobs in decreasing order of profit $p[i]$.
2. Process jobs one by one in this order. For each job, try to place it into the latest free time slot that is still at or before its deadline (search from slot $d[i]$ down towards slot 1).
3. If a free slot is found, schedule the job there (this slot is now occupied). If no free slot exists at or before its deadline, reject the job — it cannot be scheduled without missing its deadline.

Scheduling each accepted job as late as possible (rather than as early as possible) is the key insight — it leaves the maximum number of earlier, tighter slots open for jobs that are considered later in the sorted order. Proving this greedy strategy correct also uses an exchange argument: if an optimal schedule ever places a lower-profit job in a slot that a higher-profit job (considered later, but with a deadline covering that slot) could have used, swapping them can only increase or maintain total profit.

3.3 Worked Example 1

Consider 5 jobs with the following profits and deadlines:

Job	J1	J2	J3	J4	J5
Deadline $d[i]$	2	1	2	1	3
Profit $p[i]$	60	100	20	40	20

Step 1 — Sort jobs by profit, decreasing: J2 (100), J1 (60), J4 (40), J3 (20), J5 (20).

Step 2 — Process in that order. Maximum deadline is 3, so there are 3 time slots: [Slot 1] [Slot 2] [Slot 3].

Job considered	Deadline	Action	Slots after this step
J2	1	Place as late as possible $\leq 1 \rightarrow$ Slot 1 is free $\rightarrow$ place J2 in Slot 1	[J2] [] []
J1	2	Latest free slot ≤ 2 is Slot 2 $\rightarrow$ place J1 in Slot 2	[J2] [J1] []
J4	1	Latest free slot ≤ 1 is Slot 1, but it is occupied $\rightarrow$ no slot available $\rightarrow$ reject J4	[J2] [J1] []
J3	2	Latest free slot ≤ 2 is Slot 2, occupied; Slot 1 also occupied $\rightarrow$ reject J3	[J2] [J1] []
J5	3	Latest free slot ≤ 3 is Slot 3 $\rightarrow$ place J5 in Slot 3	[J2] [J1] [J5]

Result

Final schedule: Slot 1 = J2, Slot 2 = J1, Slot 3 = J5.

Jobs selected = {J2, J1, J5}. Total maximum profit = $100 + 60 + 20 = 180$.

J3 and J4 are rejected — including either would have forced some other job to miss its deadline, lowering total profit.

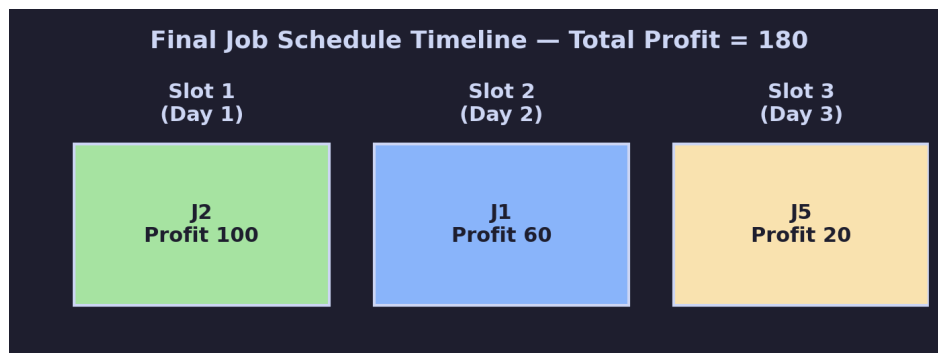


Figure 3.1 — The final accepted job schedule across the three available time slots.

3.4 Worked Example 2 (a second, independent case for practice)

Consider 7 jobs:

Job	J1	J2	J3	J4	J5	J6	J7
Deadline $d[i]$	4	1	1	1	1	3	2
Profit $p[i]$	20	10	40	30	15	25	5

Step 1 — Sort by profit, decreasing: J3 (40), J4 (30), J6 (25), J1 (20), J5 (15), J2 (10), J7 (5).

Step 2 — Maximum deadline is 4, so slots are [1][2][3][4]:

Job	Deadline	Action	Slots after
J3	1	Slot 1 free → place J3	[J3][][][]
J4	1	Slot 1 occupied → reject J4	[J3][][][]
J6	3	Latest free ≤ 3 is Slot 3 → place J6	[J3][][J6][]
J1	4	Latest free ≤ 4 is Slot 4 → place J1	[J3][][J6][J1]
J5	1	Slot 1 occupied → reject J5	[J3][][J6][J1]
J2	1	Slot 1 occupied → reject J2	[J3][][J6][J1]
J7	2	Latest free ≤ 2 is Slot 2 → place J7	[J3][J7][J6][J1]

Result

Jobs selected = {J3, J7, J6, J1}. Total maximum profit = 40 + 5 + 25 + 20 = 90.

3.5 Pseudocode

```

Algorithm JobSequencing( $d[1..n]$ ,  $p[1..n]$ )
// jobs are pre-sorted so that  $p[1] \geq p[2] \geq \dots \geq p[n]$ 
{
     $d[0] := \text{slot}[0] := 0;$            // dummy sentinel
     $k := 0;$                          // number of jobs
    scheduled so far
     $\text{slot}[1] := 1;$    $J[1] := 1;$    $k := 1;$            // schedule the first
    (highest-profit) job

    for  $i := 2$  to  $n$  do
    {
        // find the latest free slot at or before  $d[i]$ 
         $r := \min(k, d[i] - 1);$ 
        while ( $r \geq 1$  and  $\text{slot}[r]$  is occupied) do  $r := r - 1;$ 
        if ( $r \geq 1$  and slot not occupied and  $r < d[i]$ ) then
        {

```

```

        insert job i into position r+1 among scheduled jobs;
        k := k + 1;
    }
}
return the k scheduled jobs;
}

```

In practice this is often implemented using a Disjoint-Set / Union-Find structure over slots, where `find(slot)` returns the latest free slot at or before the given slot. This makes 'find the latest free slot $\leq d[i]$ ' run in near $O(1)$ amortised time, using path compression, rather than a linear scan.

3.6 Complexity Analysis

Step	Time
Sorting jobs by profit	$O(n \log n)$
Simple array scan for slot search (worst case)	$O(n^2)$
With Union-Find optimisation for slot search	$O(n \log n)$

3.7 Common Pitfalls and Exam Tips

- Always sort by profit in decreasing order first — sorting by deadline is a common mistake and gives the wrong answer.
- Remember to search for a free slot starting from $d[i]$ and moving DOWN to slot 1 (i.e. place each job as late as possible), not from slot 1 upward.
- The number of time slots needed is bounded by $\min(n, \text{max deadline})$ — you never need more slots than the largest deadline, and never more than n jobs total.
- Clearly show the slot-array state after each job is considered in exam answers — this is usually where partial marks are awarded.

3.8 Practice Questions

- Given jobs with (deadline, profit) pairs (2,100), (1,19), (2,27), (1,25), (3,15), find the optimal job sequence and total profit.
- Explain why placing each accepted job as late as possible (rather than as early as possible) is essential to the correctness of this greedy strategy.
- Describe how a Disjoint-Set Union structure can be used to optimise the slot-search step, and state the resulting time complexity.

4. Minimum-Cost Spanning Trees

4.1 What is a Spanning Tree?

Given a connected, undirected, weighted graph $G = (V, E)$ with n vertices, a spanning tree is a subgraph that (a) touches every vertex of G , (b) is connected, and (c) contains no cycles. Because it has no cycles and touches all n vertices, any spanning tree has exactly $(n - 1)$ edges. A graph can have many different spanning trees; the Minimum-cost Spanning Tree (MST) is the one whose edges sum to the smallest possible total weight.

Real-life analogy

A telecom company must lay optical fibre cable to connect n villages so that every village can communicate with every other (directly or indirectly through other villages), while spending the least amount of money on cable. Connecting every possible pair directly would be wasteful and is not even required — only enough cable to keep everything connected, with no wasteful redundant loops, is needed. That minimum-cost network of cables is precisely a Minimum-cost Spanning Tree, with villages as vertices and possible cable routes as weighted edges (weight = cost of laying that particular cable).

Two classic greedy algorithms build an MST: Prim's Algorithm (grows one connected tree outward) and Kruskal's Algorithm (adds cheapest edges globally, avoiding cycles). Both are provably optimal because the MST problem satisfies the greedy-choice property, formally captured by the 'Cut Property' of graphs, explained next.

4.2 The Cut Property (why greedy works for MST)

A 'cut' of a graph is any partition of its vertices into two non-empty disjoint sets, S and $V - S$. An edge is said to 'cross' the cut if it has one endpoint in S and the other in $V - S$. The Cut Property states: for any cut of the graph, the minimum-weight edge crossing that cut must belong to some minimum spanning tree (assuming all edge weights are distinct, the MST is unique and must contain it). This is the formal justification behind both Prim's and Kruskal's algorithms — Prim's repeatedly applies the cut property to the cut between the growing tree and the rest of the graph, while Kruskal's implicitly respects it by always taking the next cheapest edge that does not create a cycle.

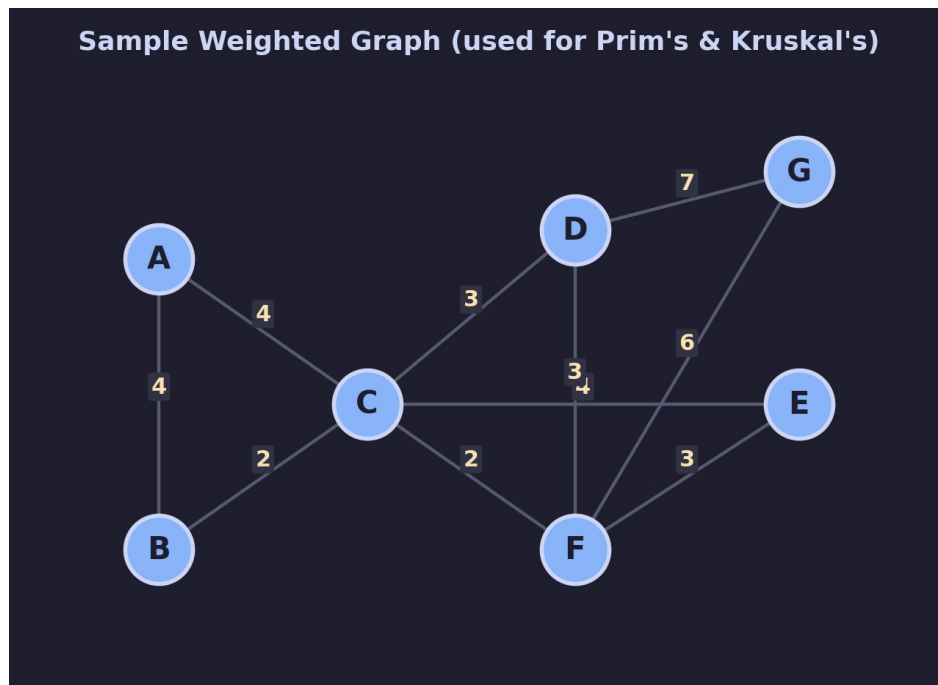


Figure 4.1 — Sample weighted graph used for both worked examples below.

4.3 Prim's Algorithm

Prim's algorithm grows the MST outward from a single starting vertex, like a snowball rolling and picking up the cheapest thing next to it at every step. At every stage, it maintains one single tree that already spans some subset of vertices, and repeatedly adds the cheapest edge that connects this growing tree to a vertex not yet in it.

Steps

1. Start with any single vertex in the tree; call this set $T = \{\text{start vertex}\}$.
2. Look at all edges that have exactly one endpoint inside T and the other endpoint outside T . Pick the cheapest such edge.
3. Add that edge and its outside vertex to T .
4. Repeat steps 2–3 until T contains all n vertices ($n - 1$ edges added).

Worked Trace (starting at vertex A)

Using the graph in Figure 4.1: A–B(4), A–C(4), B–C(2), C–D(3), C–F(2), C–E(4), D–F(3), E–F(3), F–G(6), D–G(7).

Step	Tree so far	Candidate edges crossing the cut	Edge chosen	Cost added
1	{A}	A–B(4), A–C(4)	A–B	4
2	{A,B}	A–C(4), B–C(2)	B–C	2
3	{A,B,C}	C–D(3), C–F(2), C–E(4)	C–F	2
4	{A,B,C,F}	C–D(3), D–F(3) [ignored, both in], E–F(3), F–G(6)	C–D	3
5	{A,B,C,D,F}	E–F(3), F–G(6), D–G(7)	E–F	3
6	{A,B,C,D,E,F}	F–G(6), D–G(7)	F–G	6

Result

Edges chosen: A–B, B–C, C–F, C–D, E–F, F–G. Total MST cost = $4 + 2 + 2 + 3 + 3 + 6 = 20$. This matches the tree highlighted in Figure 4.2 below.

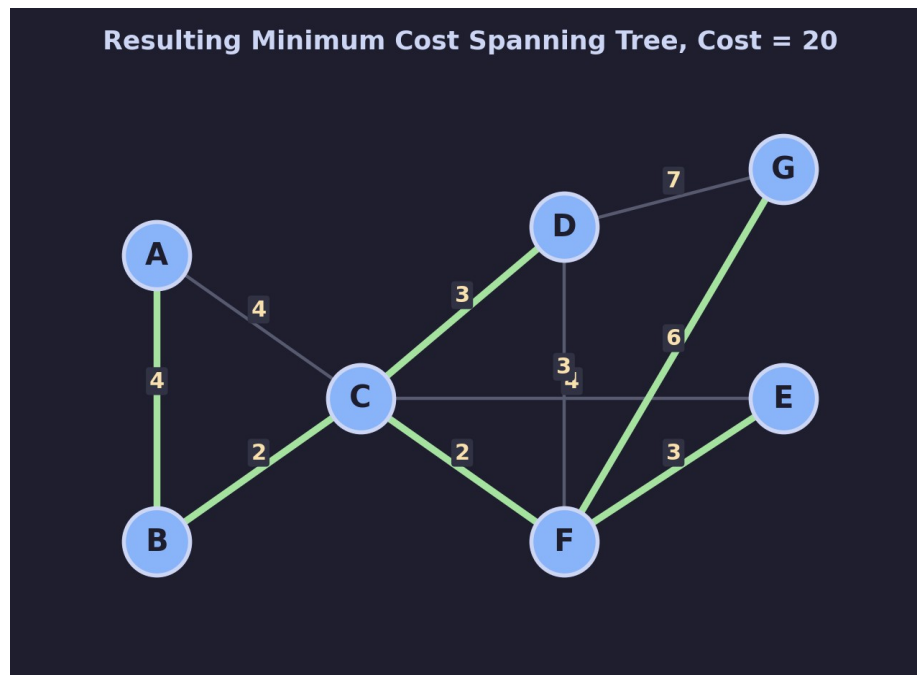


Figure 4.2 — The Minimum Cost Spanning Tree found (highlighted in green), total cost = 20.

Pseudocode (array-based, easy to hand-trace)

```

Algorithm Prim(cost[1..n][1..n])
// cost[i][j] = weight of edge (i,j), or infinity if no edge
{
    T := empty set of edges;
    near[1..n] := 1;           // near[j] = vertex in tree closest to j
    mark vertex 1 as visited; near[1] := 0;
    for i := 1 to n-1 do
    {
        let j be the unvisited vertex minimising cost[near[j]][j];
        add edge (near[j], j) to T;
        mark j as visited; near[j] := 0;
        for each unvisited vertex k do
            if cost[k][j] < cost[k][near[k]] then near[k] := j;
    }
    return T;
}

```

Why Prim's is Correct — Brief Justification

At every step, the set of vertices already in T and the remaining vertices define a cut of the graph. By the Cut Property, the minimum-weight edge crossing that cut is safe to add — it must belong to some MST. Since Prim's always selects exactly this minimum crossing edge, every edge it ever adds is safe, and after $n - 1$ additions the result is guaranteed to be a valid, minimum-cost spanning tree.

Complexity

Implementation	Time Complexity
Adjacency matrix + linear search for minimum (as above)	$O(n^2)$
Adjacency list + binary heap / priority queue	$O((n + e) \log n)$
Adjacency list + Fibonacci heap	$O(e + n \log n)$

4.4 Kruskal's Algorithm

Where Prim's grows one tree, Kruskal's works globally: it looks at every edge in the whole graph sorted from cheapest to costliest, and greedily accepts an edge as long as it does not create a cycle — regardless of which growing component that edge touches. Over time, several small tree fragments merge into fewer, larger fragments, until only one tree spanning all vertices remains.

Real-life analogy

Imagine several separate friend groups (islands of people who don't yet know anyone outside their own group) and a matchmaking event whose goal is to connect everyone into a single social network using the fewest, cheapest introductions. The organiser sorts every possible introduction by cost (e.g. travel expense to arrange the meeting) and, cheapest first, approves an introduction only if the two people belong to different groups — an introduction between two people already in the same group would be a wasted, redundant expense (it would create a cycle, not add new connectivity).

Steps

1. Sort all e edges of the graph in non-decreasing order of weight.
2. Initialise n separate components, one per vertex (a Disjoint-Set / Union-Find structure is ideal here).
3. Scan edges in sorted order. For each edge (u, v) : if u and v are in different components, accept the edge (union the two components); otherwise reject it (it would form a cycle).
4. Stop once $(n - 1)$ edges have been accepted.

Worked Trace (same graph as Figure 4.1)

Sorted edges by weight: B–C(2), C–F(2), C–D(3), D–F(3), E–F(3), A–B(4), A–C(4), C–E(4), F–G(6), D–G(7).

Edge examined	Weight	Components before	Decision
B–C	2	B and C in different sets	Accept
C–F	2	C and F in different sets	Accept
C–D	3	C and D in different sets	Accept
D–F	3	D and F now in the SAME set (via C)	Reject — would form a cycle
E–F	3	E and F in different sets	Accept
A–B	4	A and B in different sets	Accept
A–C	4	A and C now in the SAME set	Reject — would form a cycle
C–E	4	C and E now in the SAME set	Reject — would form a cycle
F–G	6	F and G in different sets	Accept — 6 edges reached, stop

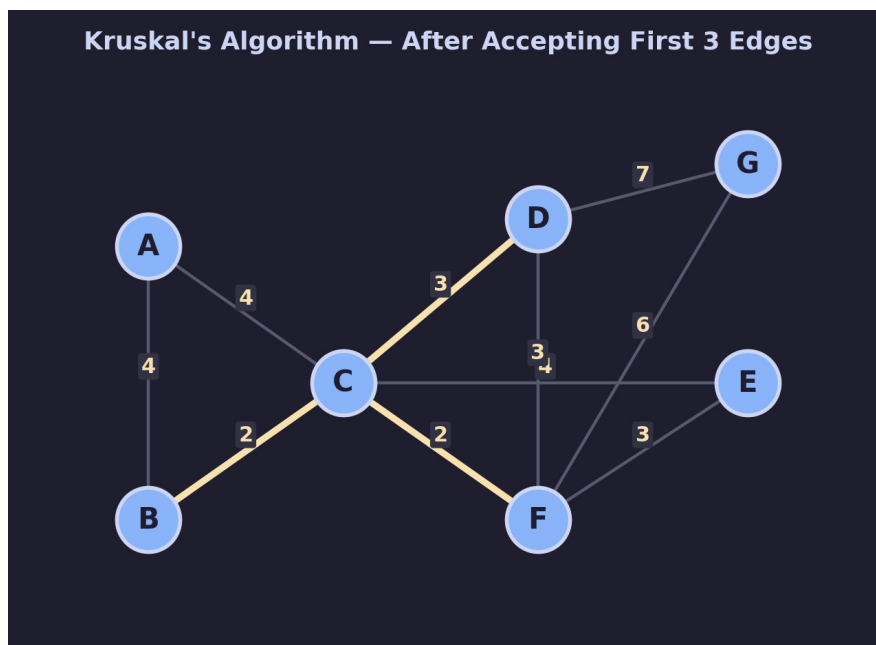


Figure 4.3 — Kruskal's Algorithm partway through: the first 3 accepted edges (yellow) form two growing tree fragments before D–F is rejected as a cycle.

Result

Edges accepted: B–C, C–F, C–D, E–F, A–B, F–G — the same 6 edges Prim's found (a graph's MST edge-set is unique here because all weights that matter are distinct at each decision point). Total cost = $2 + 2 + 3 + 3 + 4 + 6 = 20$.

Notice that Kruskal's built the tree in a completely different order from Prim's — it never even touched vertex A until step 6 — yet both converge to a spanning tree of the same minimum cost, 20. This is expected: whenever the weights that matter for the decision are distinct, the MST is unique regardless of which correct algorithm builds it.

Pseudocode

```

Algorithm Kruskal(E[1..e], n)
// E is the edge list, each with a weight; n = number of vertices
{
    sort E in non-decreasing order of weight;
    T := empty set;
    make n singleton disjoint sets, one per vertex;
    count := 0;
    for each edge (u,v) in sorted E, while count < n-1 do
    {
        if FIND(u) != FIND(v) then          // different components?
        {
            T := T + {(u,v)};
            UNION(u, v);
            count := count + 1;
        }
    }
    return T;
}

```

Disjoint-Set Union (DSU) — the engine behind Kruskal's

FIND(x) returns a representative label for the component containing x; UNION(x, y) merges the two components containing x and y into one. A naive DSU can take $O(n)$ per operation, but with two standard optimisations — union by rank (always attach the smaller tree under the root of the larger one) and path compression (during FIND, make every visited node point directly to the root) — the amortised cost of each operation drops to nearly $O(1)$, specifically $O(\alpha(n))$, where α is the extremely slow-growing inverse Ackermann function (effectively a constant for all realistic input sizes).

Complexity

Step	Time
Sorting all e edges	$O(e \log e)$
e FIND/UNION operations (with path compression + union by rank)	$O(e \cdot \alpha(n)) \approx O(e)$
Overall	$O(e \log e)$, i.e. $O(e \log n)$ since $e < n^2$

4.5 Prim's vs Kruskal's — Side-by-Side

Aspect	Prim's Algorithm	Kruskal's Algorithm
Growth style	One tree grows outward from a start vertex	Many fragments merge together globally
Best suited for	Dense graphs (adjacency matrix)	Sparse graphs (edge list + Union-Find)
Key data structure	Priority queue / min-array over vertices	Sorted edge list + Disjoint-Set Union
Cycle avoided by	Never picking an edge with both ends	Explicit FIND check before accepting

Aspect	Prim's Algorithm	Kruskal's Algorithm
	already inside the tree	an edge
Typical complexity	$O(n^2)$ or $O(e \log n)$ with heaps	$O(e \log e)$

4.6 Common Pitfalls and Exam Tips

- In Prim's, remember the tree must stay connected at every intermediate step — this is the defining difference from Kruskal's, where fragments can be scattered before finally joining.
- In Kruskal's, always check with FIND whether both endpoints are already in the same component before accepting an edge — forgetting this check is the most common mistake and produces a graph with a cycle, not a tree.
- If the graph is disconnected, no spanning tree exists at all — both algorithms should be understood to require a connected input graph.
- When several edges tie for the same minimum weight, any valid choice among them still yields a correct (though possibly different-looking) MST of the same total cost.

4.7 Practice Questions

- Using Prim's algorithm starting from vertex C, find the MST for the graph in Figure 4.1 and state its total cost.
- Trace Kruskal's algorithm on a graph with edges (A,B,1),(B,C,2),(A,C,3),(C,D,4),(B,D,5) and list the accepted/rejected edges in order.
- Explain the Cut Property and how it justifies the correctness of both Prim's and Kruskal's algorithms.
- State the time complexity of Kruskal's algorithm when implemented with (a) a naive Union-Find and (b) a Union-Find with path compression and union by rank.

5. An Optimal Randomized Algorithm

5.1 Why bring randomness into a Greedy chapter?

Every algorithm seen so far is deterministic — given the same input, it always makes the same sequence of choices and takes the same amount of time. A randomized algorithm deliberately uses a source of random numbers as part of its logic, so that its behaviour (and sometimes its running time) can differ across runs on the very same input. This unit introduces the idea because several greedy-style selection problems — most notably finding an order statistic (like the median, or the k -th smallest element of an unsorted list) — have an elegant, provably efficient randomized greedy strategy that mirrors the partitioning idea used in Quicksort.

Real-life analogy

Suppose you must find roughly the middle-priced house among thousands of unsorted property listings, and sorting all of them first would be slow. Instead of a fixed, predictable strategy, you randomly flip open a listings booklet to one page, use that listing's price as a rough dividing line, and quickly separate the remaining listings into 'cheaper' and 'costlier' piles — repeating on whichever pile must contain the middle-priced house. Because your dividing line is chosen randomly rather than always being (say) the first listing on the page, an adversary who deliberately arranged the listings to trip up your strategy cannot reliably slow you down. This is the core idea of Randomized Quickselect.

5.2 Two Flavours of Randomized Algorithms

Type	Guarantee	Example
Las Vegas	Always gives the correct answer; only the running time is random	Randomized Quicksort, Randomized Quickselect
Monte Carlo	Runs in fixed/bounded time; has a small, controllable probability of an incorrect answer	Randomized primality testing (Miller–Rabin)

The algorithm studied in this section — Randomized Selection — is a Las Vegas algorithm: it always returns the correct k -th smallest element; what varies from run to run is only how long it takes.

5.3 Worked Idea: Randomized Selection (finding the k-th smallest element)

Given an unsorted array of n numbers, the goal is to find the k -th smallest element (e.g. the median) without fully sorting the array — full sorting would cost $O(n \log n)$, more work than necessary.

1. Pick a random element from the array; call it the pivot.
2. Partition the array around the pivot: all elements smaller than the pivot go to its left, all larger go to its right (this is the same partition step used in Quicksort).
3. Let the pivot land at position m after partitioning. If m equals k , the pivot itself is the answer — stop. If $k < m$, recurse only on the left part. If $k > m$, recurse only on the right part.

Because the pivot is random, no fixed 'bad' input can consistently force worst-case behaviour — on average, each recursive call discards roughly half the remaining elements, giving an expected running time of $O(n)$, which is provably optimal (you cannot find the k -th smallest element in less than linear time, because every element must at least be looked at once).

5.4 Worked Numeric Trace

Array: [12, 3, 5, 7, 19, 26, 21, 9] ($n = 8$). Find the $k = 3$ rd smallest element.

Call	Sub-array considered	Random pivot chosen	Partition result	Pivot's rank m	Next action
1	[12,3,5,7,19,26,21,9]	9	[3,5,7] 9 [12,19,26,21]	4	$k=3 < m=4 \rightarrow$ recurse LEFT on [3,5,7], still looking for rank 3
2	[3,5,7]	5	[3] 5 [7]	2	$k=3 > m=2 \rightarrow$ recurse RIGHT on [7], now looking for rank $3-2=1$
3	[7]	—	single element, base case	1	only element left $\rightarrow$ return 7

Each recursive call narrows the search to one side of the partition and adjusts k accordingly (k stays the same when recursing left, but becomes $k - m$ when recursing right, since m elements — and the pivot — have been eliminated from consideration on the left).

Result

The 3rd smallest element of [12, 3, 5, 7, 19, 26, 21, 9] is 7. A full sort would have produced [3, 5, 7, 9, 12, 19, 21, 26], confirming that 7 is indeed at position 3.

5.5 Pseudocode

```

Algorithm RandomizedSelect(A, low, high, k)
// returns the k-th smallest element of A[low..high]
{
    if (low == high) then return A[low];
    p := RandomPartition(A, low, high);    // randomly-chosen pivot,
partitioned
    m := p - low + 1;                      // rank of pivot within
A[low..high]
    if (k == m) then return A[p];
    else if (k < m) then
        return RandomizedSelect(A, low, p-1, k);
    else
        return RandomizedSelect(A, p+1, high, k-m);
}

Algorithm RandomPartition(A, low, high)
{
    i := random integer between low and high;
    swap A[i] and A[high];                // move random pivot to the end
    return Partition(A, low, high);       // standard Lomuto/Hoare
partition
}

```

5.6 Why this is called 'optimal'

Optimality argument

$O(n)$ expected time matches the trivial lower bound of $O(n)$ (you must at minimum examine every element once to be sure none of them is the answer), so — on average — this randomized greedy-partition strategy cannot be beaten asymptotically. This is what 'optimal randomized algorithm' refers to in this context: an expected-linear-time algorithm for selection, first analysed by Hoare (as 'Quickselect') and later refined by Blum, Floyd, Pratt, Rivest and Tarjan into a version (the 'median of medians' algorithm) with a guaranteed (not just expected) worst-case linear time, at the cost of a larger constant factor.

Complexity

Case	Time
Expected (average) case	$O(n)$
Worst case (extremely unlucky pivot choices every time)	$O(n^2)$
Deterministic worst-case version (median-of-medians)	$O(n)$, guaranteed
Space	$O(1)$ extra (in-place partitioning)

5.7 Comparing Deterministic vs Randomized Selection

Aspect	Deterministic Selection (median-of-medians)	Randomized Selection
Worst-case time	$O(n)$, guaranteed	$O(n^2)$, extremely unlikely
Average-case time	$O(n)$	$O(n)$, with a smaller constant factor
Implementation complexity	More complex (groups of 5, recursive median-finding)	Simple (one random pivot pick per call)
Used in practice	Rarely, due to overhead	Frequently, e.g. in library implementations of selection

5.8 Common Pitfalls and Exam Tips

- Do not confuse 'randomized' with 'probabilistic correctness' — Randomized Selection is a Las Vegas algorithm and always gives the exact correct answer; only its running time varies.
- Be ready to state clearly why the worst case is $O(n^2)$: this happens only if the random pivot is unluckily always the minimum or maximum element, at every single recursive call — a vanishingly small probability for large n .
- Exam questions often ask you to trace one or two levels of recursion by hand with a given (or arbitrarily chosen) pivot — show the partition step explicitly.
- Remember the key comparison point: full sorting costs $O(n \log n)$; selection only needs $O(n)$, because we discard one whole side of the partition at each step instead of recursively sorting both sides.

5.9 Practice Questions

- Explain the difference between Las Vegas and Monte Carlo randomized algorithms, with one example of each.
- Trace Randomized Quickselect to find the 4th smallest element of the array $[8, 4, 2, 9, 1, 7, 5]$, choosing pivots as the last element of each sub-array at every step.
- Derive why the expected running time of Randomized Selection is $O(n)$, briefly explaining the recurrence relation involved.
- Why can Randomized Selection not be improved asymptotically below $O(n)$? Justify with a lower-bound argument.

6. Optimal Merge Patterns

6.1 Problem Statement

Suppose you have several sorted files of different lengths that must all be merged into one single sorted file, but you can only merge two files at a time (a 2-way merge). Merging two files of length p and q costs $p + q$ comparisons/moves (roughly). Since there are several possible orders in which to perform the pairwise merges, the total cost varies depending on that order. The goal is to choose the merge order that minimises the total number of record moves across all merge steps.

Real-life analogy

Imagine several classes of students of different sizes that must eventually be combined into a single large class, but the school hall can only physically merge two classes at a time (call the roll, re-seat everyone, issue new class lists — an amount of work proportional to the total number of students being merged in that step). Merging the two smallest classes first, again and again, means that the largest classes only ever get 'moved' (re-processed) the fewest possible number of times, while small classes — which are cheap to move — get shuffled around more often. This intuition, always combine the two smallest available piles, is exactly the optimal merge strategy, and it is identical in spirit to building a Huffman coding tree.

6.2 The Greedy Strategy

1. Maintain a collection of file sizes, starting with the n given file lengths.
2. Repeatedly pick the two smallest sizes currently in the collection, merge them (cost = sum of the two sizes), remove both from the collection and insert the merged size back in.
3. Repeat until only one size remains — that size is 0 cost by itself; the total of all the individual merge costs recorded along the way is the answer.

A min-heap (priority queue) is the natural data structure for always retrieving 'the two smallest sizes' efficiently — this mirrors exactly how a Huffman coding tree is built from character frequencies.

6.3 Worked Example 1

Five sorted files of lengths 2, 3, 5, 7, 9 must be merged into one file at minimum total cost.

Step	Two smallest sizes picked	Merge cost	Collection after this step
1	2 and 3	5	{5, 5, 7, 9}
2	5 and 5	10	{7, 9, 10}
3	7 and 9	16	{10, 16}
4	10 and 16	26	{26} ← done

Result

Total merge cost = $5 + 10 + 16 + 26 = 57$. No other pairing order produces a lower total — for example, merging $2+3=5$ then immediately $5+7=12$ then $12+5=17$ then $17+9=26$ gives total cost $5+12+17+26 = 60$, which is worse.

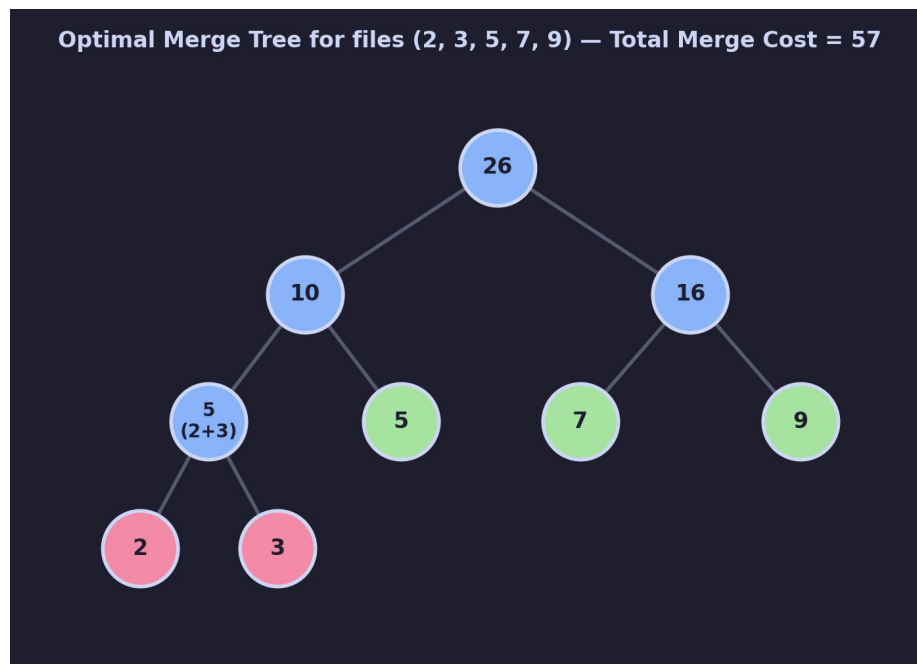


Figure 6.1 — The optimal 2-way merge tree for files (2, 3, 5, 7, 9). Every internal node's value is the cost of that merge step; the sum of all internal nodes (57) is the total cost.

Connection to Weighted External Path Length

If each file is placed as a leaf of a binary merge tree (as drawn in Figure 6.1), the total merge cost exactly equals the sum of (file length $\times$ depth of that leaf in the tree) — also called the tree's Weighted External Path Length. Minimising total merge cost is therefore identical to minimising weighted external path length, which is the very same problem Huffman coding solves for building optimal prefix codes. In fact, the algorithm above IS Huffman's algorithm, just applied to file sizes instead of character frequencies.

Verifying with weighted path length for Figure 6.1: leaves are 2 (depth 3), 3 (depth 3), 5 (depth 2), 7 (depth 2), 9 (depth 2). Weighted path length = $2 \times 3 + 3 \times 3 + 5 \times 2 + 7 \times 2 + 9 \times 2 = 6 + 9 + 10 + 14 + 18 = 57$ — matching the total merge cost exactly.

6.4 Worked Example 2 (a second, independent case for practice)

Six sorted files of lengths 4, 6, 8, 2, 10, 5 must be merged.

Step	Sorted collection before this step	Two smallest picked	Merge cost	Collection after
1	{2,4,5,6,8,10}	2 and 4	6	{5,6,6,8,10}
2	{5,6,6,8,10}	5 and 6	11	{6,8,10,11}
3	{6,8,10,11}	6 and 8	14	{10,11,14}
4	{10,11,14}	10 and 11	21	{14,21}
5	{14,21}	14 and 21	35	{35} ← done

Result

Total merge cost = $6 + 11 + 14 + 21 + 35 = 87$.

6.5 Pseudocode

```

Algorithm OptimalMerge(size[1..n])
// size[i] = length of the i-th sorted file
{
    build a min-heap H from size[1..n];
    totalCost := 0;
    while (H has more than 1 element) do
    {
        a := extractMin(H);
        b := extractMin(H);
        merged := a + b;
        totalCost := totalCost + merged;
        insert(H, merged);
    }
    return totalCost;
}

```

Complexity

Step	Time
Building the initial min-heap	$O(n)$
$n - 1$ rounds, each doing 2 extractMin + 1 insert ($O(\log n)$ each)	$O(n \log n)$
Overall	$O(n \log n)$

6.6 Two-Way vs k-Way Merge Patterns

The version above is the 2-way merge pattern (only two files merged at a time). A generalisation, the k -way merge pattern, allows merging k files simultaneously at each step, at a cost equal to the sum of all k file sizes being merged. The same greedy insight extends naturally: repeatedly merge the k smallest available files. k -way merge patterns are common in external sorting, where k is chosen based on how many files can be kept open simultaneously (limited by available memory buffers).

6.7 Common Pitfalls and Exam Tips

- Always merge the two SMALLEST available sizes at every step — merging any other pair, even if it looks convenient, generally increases the total cost.
- Remember to re-insert the merged size back into the collection before picking the next pair — it competes with the remaining original sizes on equal footing.
- Drawing the merge tree (as in Figure 6.1) and computing weighted path length is a useful way to double-check your total cost in an exam.
- This problem is mathematically identical to Huffman coding — if the syllabus later covers Huffman coding, you can directly reuse this greedy strategy and its optimality proof.

6.8 Practice Questions

- Given files of lengths 20, 30, 10, 5, 30, find the optimal merge pattern and total merge cost, showing the merge tree.
- Explain the equivalence between the Optimal Merge Pattern problem and Huffman coding.
- Derive the time complexity of the greedy Optimal Merge algorithm when implemented with a min-heap.
- If k -way merging is allowed instead of 2-way, briefly describe how the greedy strategy changes.

7. Single Source Shortest Paths — Dijkstra's Algorithm

7.1 Problem Statement

Given a directed (or undirected) weighted graph with non-negative edge weights and a single designated source vertex, find the shortest-path distance from the source to every other vertex in the graph. 'Shortest' means the path whose edge weights sum to the smallest total.

Real-life analogy

This is exactly what a GPS navigation app does when it computes 'fastest route' from your current location to every reachable landmark: roads are edges, junctions are vertices, and travel time is the edge weight. Dijkstra's algorithm mimics an ever-expanding 'wave' spreading outward from the starting point — the wave always finalises the closest not-yet-finalised junction next, because any junction that is currently closest cannot possibly be reached more cheaply later through some other, currently-farther junction (this only holds because all road travel times are non-negative — Dijkstra breaks down if negative weights, like a toll rebate, are allowed).

7.2 The Greedy Strategy

1. Maintain a distance estimate $\text{dist}[v]$ for every vertex, initialised to ∞ for all vertices except the source, which is 0.
2. Maintain a set S of vertices whose shortest distance from the source is already finalised (known to be correct); initially S is empty.
3. Repeatedly pick the vertex u NOT in S with the smallest $\text{dist}[u]$ value; this greedy pick is guaranteed correct once chosen (this is the greedy-choice property for this problem), so add u to S .
4. Relax every edge (u, v) leaving u : if $\text{dist}[u] + \text{weight}(u, v) < \text{dist}[v]$, update $\text{dist}[v]$ to this smaller value.
5. Repeat steps 3–4 until every vertex has been added to S .

7.3 Why the Greedy Choice is Safe — Proof Sketch

Claim: when u is chosen (the not-yet-finalised vertex with minimum $\text{dist}[u]$), $\text{dist}[u]$ is already the true shortest distance from the source. Proof by contradiction: suppose the true shortest path to u were actually shorter, going through some other not-yet-finalised vertex y ($\text{dist}[y] < \text{true distance}$ would be needed along the way). But y is not in S , and since all edge weights are non-negative, the moment the path reaches y its accumulated length is already at least $\text{dist}[y] \geq \text{dist}[u]$ (because u was chosen as the minimum among not-yet-finalised vertices). Continuing from y to u can only add more non-negative weight, so this alternate path cannot possibly be shorter than $\text{dist}[u]$. Hence $\text{dist}[u]$ must already be optimal. This argument critically depends on non-negative weights — with a negative edge, a path could 'save' distance after passing through y , breaking the whole argument.

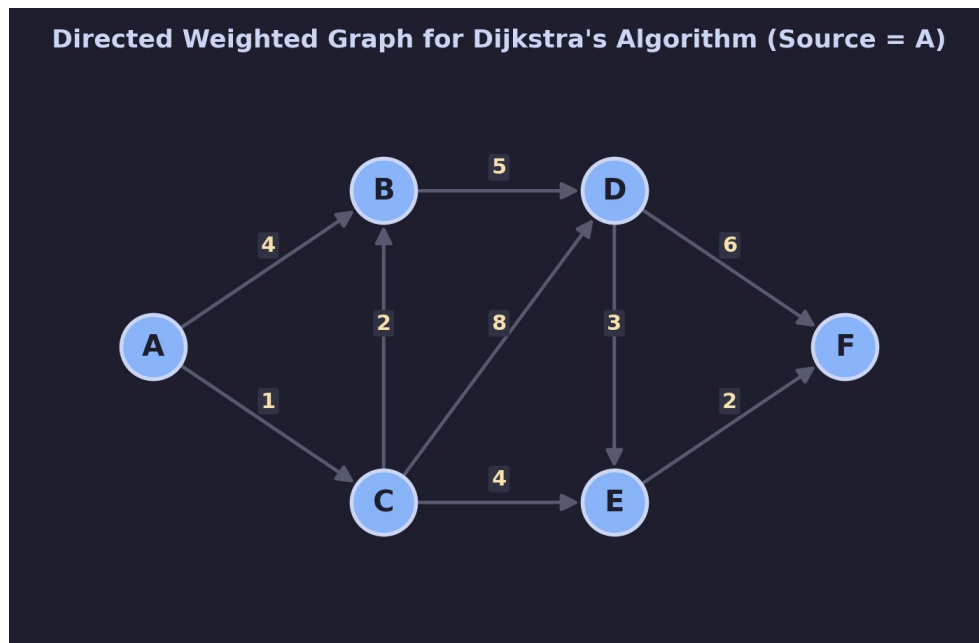


Figure 7.1 — Directed weighted graph; shortest paths are computed from source A.

7.4 Worked Trace 1 (source = A)

Edges: $A \rightarrow B(4)$, $A \rightarrow C(1)$, $C \rightarrow B(2)$, $B \rightarrow D(5)$, $C \rightarrow D(8)$, $C \rightarrow E(4)$, $D \rightarrow E(3)$, $D \rightarrow F(6)$, $E \rightarrow F(2)$.

Iteration	Vertex finalised (added to S)	dist[A]	dist[B]	dist[C]	dist[D]	dist[E]	dist[F]
Initial	—	0	∞	∞	∞	∞	∞
1	A (dist 0)	0	4	1	∞	∞	∞
2	C (dist 1)	0	3	1	9	5	∞
3	B (dist 3)	0	3	1	8	5	∞
4	E (dist 5)	0	3	1	8	5	7
5	F (dist 7)	0	3	1	8	5	7
6	D (dist 8)	0	3	1	8	5	7

Notes on the key updates: after finalising A, edge $A \rightarrow C(1)$ gives $\text{dist}[C]=1$, and $A \rightarrow B(4)$ gives $\text{dist}[B]=4$. After finalising C (the smallest tentative distance, 1), relaxing $C \rightarrow B(2)$ improves $\text{dist}[B]$ from 4 to $1+2=3$, and $C \rightarrow E(4)$ sets $\text{dist}[E]=1+4=5$; $C \rightarrow D(8)$ sets $\text{dist}[D]=1+8=9$. After finalising B (dist 3), relaxing $B \rightarrow D(5)$ improves $\text{dist}[D]$ from 9 to $3+5=8$. After finalising E (dist 5), relaxing $E \rightarrow F(2)$ sets $\text{dist}[F]=5+2=7$.

Final Result

Shortest distances from A: $A=0$, $B=3$ (via $A \rightarrow C \rightarrow B$), $C=1$ (direct), $D=8$ (via $A \rightarrow C \rightarrow B \rightarrow D$), $E=5$ (via $A \rightarrow C \rightarrow E$), $F=7$ (via $A \rightarrow C \rightarrow E \rightarrow F$).

7.5 Worked Trace 2 (a second, independent case for practice)

Consider a graph with vertices $\{S, A, B, C, D\}$ and directed edges: $S \rightarrow A(10)$, $S \rightarrow C(3)$, $A \rightarrow C(1)$, $C \rightarrow A(4)$, $A \rightarrow B(2)$, $C \rightarrow D(8)$, $C \rightarrow B(2)$, $B \rightarrow D(7)$, $D \rightarrow B(9)$, $D \rightarrow S(7)$.

Iteration	Vertex finalised	dist[S]	dist[A]	dist[B]	dist[C]	dist[D]
Initial	—	0	∞	∞	∞	∞
1	S (0)	0	10	∞	3	∞
2	C (3)	0	7	5	3	11
3	B (5)	0	7	5	3	11
4	A (7)	0	7	5	3	11
5	D (11)	0	7	5	3	11

Key relaxations: from S, edges give $\text{dist}[A]=10$, $\text{dist}[C]=3$. Finalising C (smallest, 3): $C \rightarrow A(4)$ gives $3+4=7 < 10$, so $\text{dist}[A]$ updates to 7; $C \rightarrow D(8)$ gives $\text{dist}[D]=11$; $C \rightarrow B(2)$ gives $\text{dist}[B]=5$. Finalising B (5): $B \rightarrow D(7)$ gives $5+7=12$, not better than 11, no update. Finalising A (7): $A \rightarrow B(2)$ gives $7+2=9$, not better than 5; $A \rightarrow C(1)$ gives $7+1=8$, not better than 3. Finalising D (11): $D \rightarrow B(9)$ gives $11+9=20$, no improvement; $D \rightarrow S(7)$ gives 18, S already finalised at 0.

Final Result

Shortest distances from S: $S=0$, $A=7$ (via $S \rightarrow C \rightarrow A$), $B=5$ (via $S \rightarrow C \rightarrow B$), $C=3$ (direct), $D=11$ (via $S \rightarrow C \rightarrow D$).

7.6 Pseudocode

```

Algorithm Dijkstra(cost[1..n][1..n], source)
// cost[i][j] = weight of edge i->j, or infinity if no edge; non-negative
// weights only
{
    for v := 1 to n do { dist[v] := infinity; S[v] := false; }
    dist[source] := 0;
    for i := 1 to n do
    {
        u := the vertex not in S with minimum dist[u];
        S[u] := true;                // finalise u
        for each vertex v not in S do
            if dist[u] + cost[u][v] < dist[v] then
                dist[v] := dist[u] + cost[u][v];    // relax edge (u,v)
    }
    return dist[];
}

```

7.7 Complexity Analysis

Implementation	Time Complexity
Adjacency matrix + linear scan for minimum (as above)	$O(n^2)$
Adjacency list + binary heap priority queue	$O((n + e) \log n)$
Adjacency list + Fibonacci heap	$O(e + n \log n)$

Caution

Dijkstra's algorithm assumes all edge weights are non-negative. If the graph can contain negative weights, the greedy-choice property breaks down (a vertex finalised early might later be reachable more cheaply through a negative-weight edge), and the Bellman–Ford algorithm — which is Dynamic Programming based, not greedy — must be used instead. Bellman-Ford runs in $O(n \cdot e)$ time but correctly handles negative weights (and can detect negative-weight cycles, for which no shortest path is even well-defined).

7.8 Dijkstra's vs Bellman-Ford — Quick Comparison

Aspect	Dijkstra's Algorithm	Bellman-Ford Algorithm
Approach	Greedy	Dynamic Programming
Negative weights	Not allowed	Allowed (also detects negative cycles)
Time complexity	$O(n^2)$ or $O((n+e) \log n)$ with heaps	$O(n \cdot e)$
Typical use case	Road networks, most real-world graphs	Graphs where negative weights are possible (e.g. currency arbitrage)

7.9 Common Pitfalls and Exam Tips

- Always double-check that the graph has non-negative weights before applying Dijkstra's — this is a very common exam trick question.
- When hand-tracing, maintain a clear table of $\text{dist}[]$ values after each iteration (as shown above) — most marks are awarded for correctly showing the relaxation steps, not just the final answer.
- Remember that once a vertex is finalised (added to S), its $\text{dist}[]$ value never changes again — this is the core greedy commitment.
- Dijkstra's finds shortest paths from a SINGLE source to ALL other vertices; if all-pairs shortest paths are needed instead, the Floyd-Warshall algorithm (Dynamic Programming) is typically used.

7.10 Practice Questions

- Using Dijkstra's algorithm, find the shortest distances from vertex A in a graph with edges $A \rightarrow B(2)$, $A \rightarrow C(5)$, $B \rightarrow C(1)$, $B \rightarrow D(4)$, $C \rightarrow D(1)$, $D \rightarrow E(3)$, $C \rightarrow E(7)$.
- Explain, with a proof sketch, why Dijkstra's algorithm fails to give correct results when negative edge weights are present.
- Compare Dijkstra's algorithm and Bellman-Ford's algorithm in terms of approach, time complexity, and applicability.
- State the time complexity of Dijkstra's algorithm using (a) an adjacency matrix and (b) an adjacency list with a binary heap, and explain the difference.